Agenda

- → How to approach Schema Design ✓
- → How to code ✓
- → Design TTT

How to approach Schema Design

- 1:1 ⇒ put id of 1 side on other side
- 1:M or M:1 ⇒ put id of m side in 1 side
- M:M ⇒ Mapping Table

Step 1: For all the classes, create a table

Step 2: For each class, put its primitive attributes (int / String) Not enums / Not other class as columns in the table.

```
class Ticket {
    int id;
    Date timeOfBooking;
    List<Seat>
    User bookedBy;
    int amount;
}
```

Tickets

id	timeOfBook	amount

Step 3 For every relation ⇒

a.) Find cardinality of rel^n

b.) Based on cardinality, rep the rel^n

⇒ Parking lot

How to Code

a.) Project Structure (Structure your codebase)

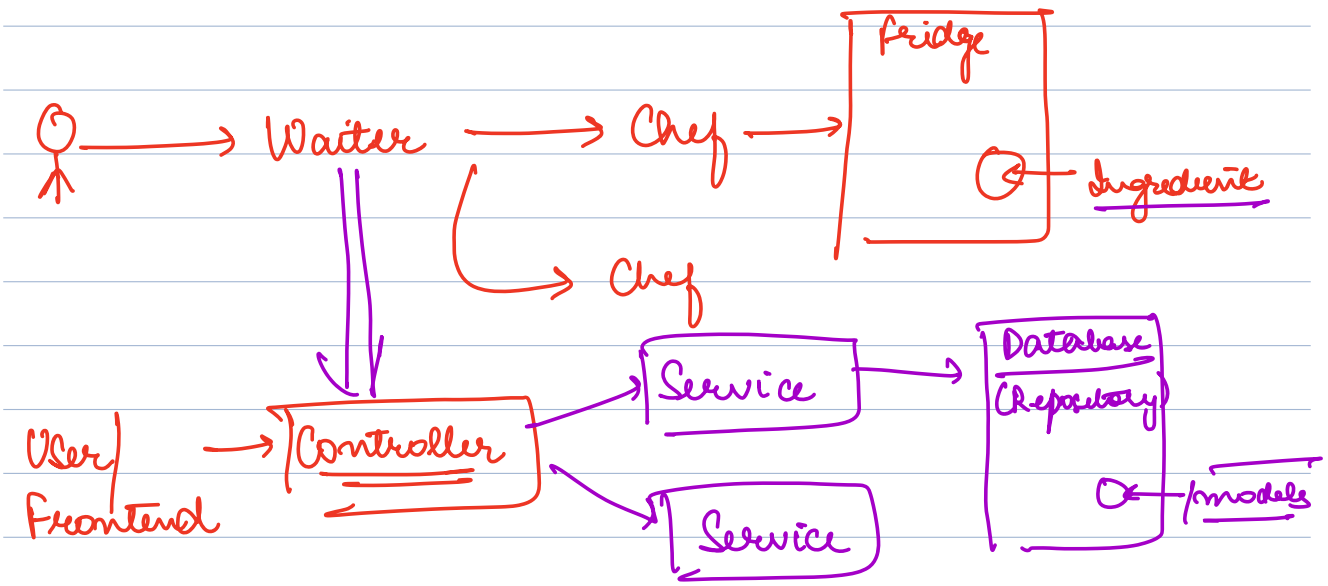
b.) Atleast something must be running

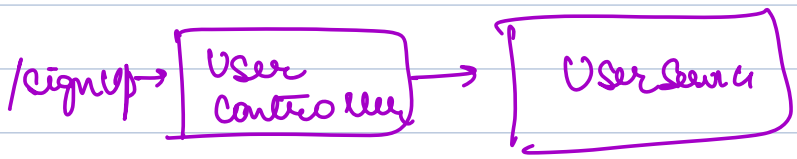
Models → Controller → Service

① Code all the models (classes in the class diagram)

② Go req by req:

→ code all classes etc that might be needed to imp the functionality





models/

all classes in class diag

(dao) repositories/

data access object

all classes that query DB

Services/

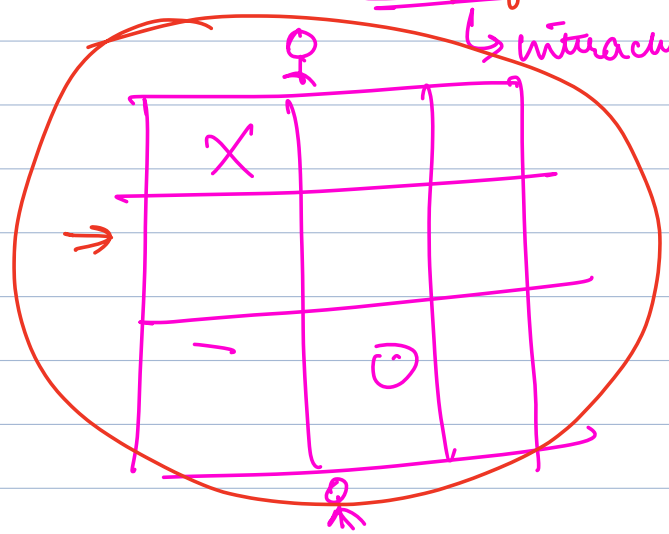
controllers /

DESIGN TICTACTOE

Overview

- you know
- you don't know

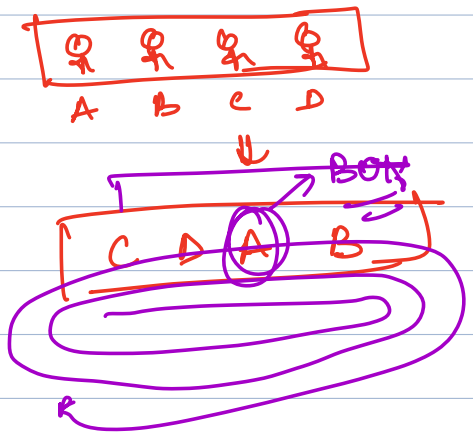
→ align
→ type of system
→ interactive



Gathering Req

- ⇒ Size of board can be $n \times n$
- ⇒ # of players ⇒ $n-1$. Every player will have their own symbol.
- ⇒ Every player can choose their symbol as any char at start of game. No 2 people should have same.
- ⇒ At the start of game we randomize list

and play in the order



⇒ Will the game have bots ⇒ Yes

Only 1 bot per game

Rep Bots

⇒ Will bots have diff difficulty level

⇒ EASY, MEDIUM, HARD

→ Timer b/w moves ✗

→ Game ends as soon as someone has won or game is drawn

→ Tournaments ✗

→ Pause / Resuming / Exit ✗

→ Leaderboard ✗

⇒ How to decide victory

⇒ Multiple diff ways in which someone can win

A	B	C	D
row	col	diag	corner

⇒ At the start of the game, we decide all the ways we want to support

Game 1

A D

Game 2

A B C D

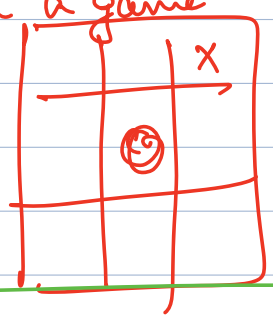
⇒ Atleast 2 players

⇒ Blocked Cells

⇒ Allow people can undo moves

⇒ any # of undo can happen

⇒ Rewatch a game



Any game

- → Size of board
- → # players
- → Bots & Diff level
- → Leaderboard
- → Undo
- → How will game start
- → How will game end
- → Rewatch a game after ends

Clarify Req

~~Working of Behaviour~~

whose turn →

when will someone win

how will game start

how many undo can be made

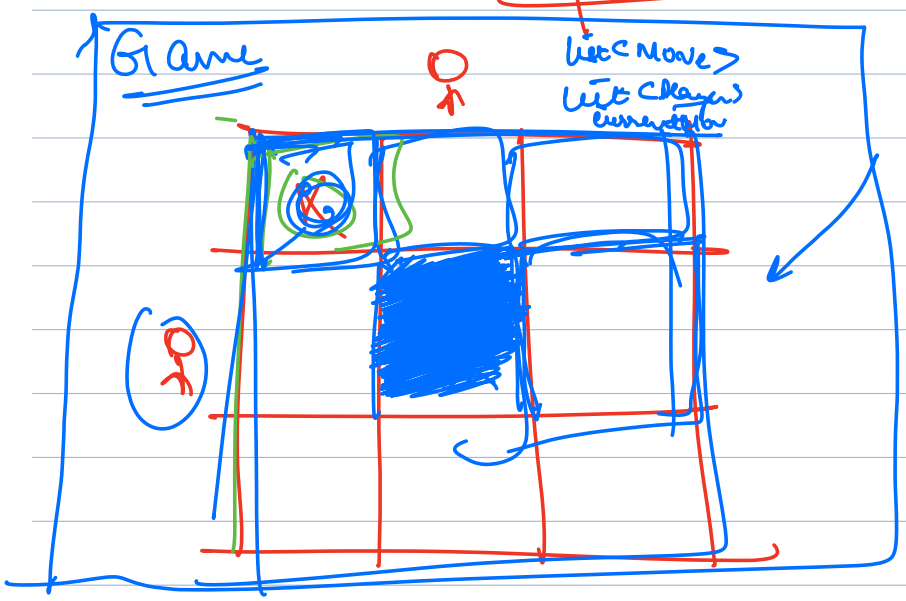
Class Diagram

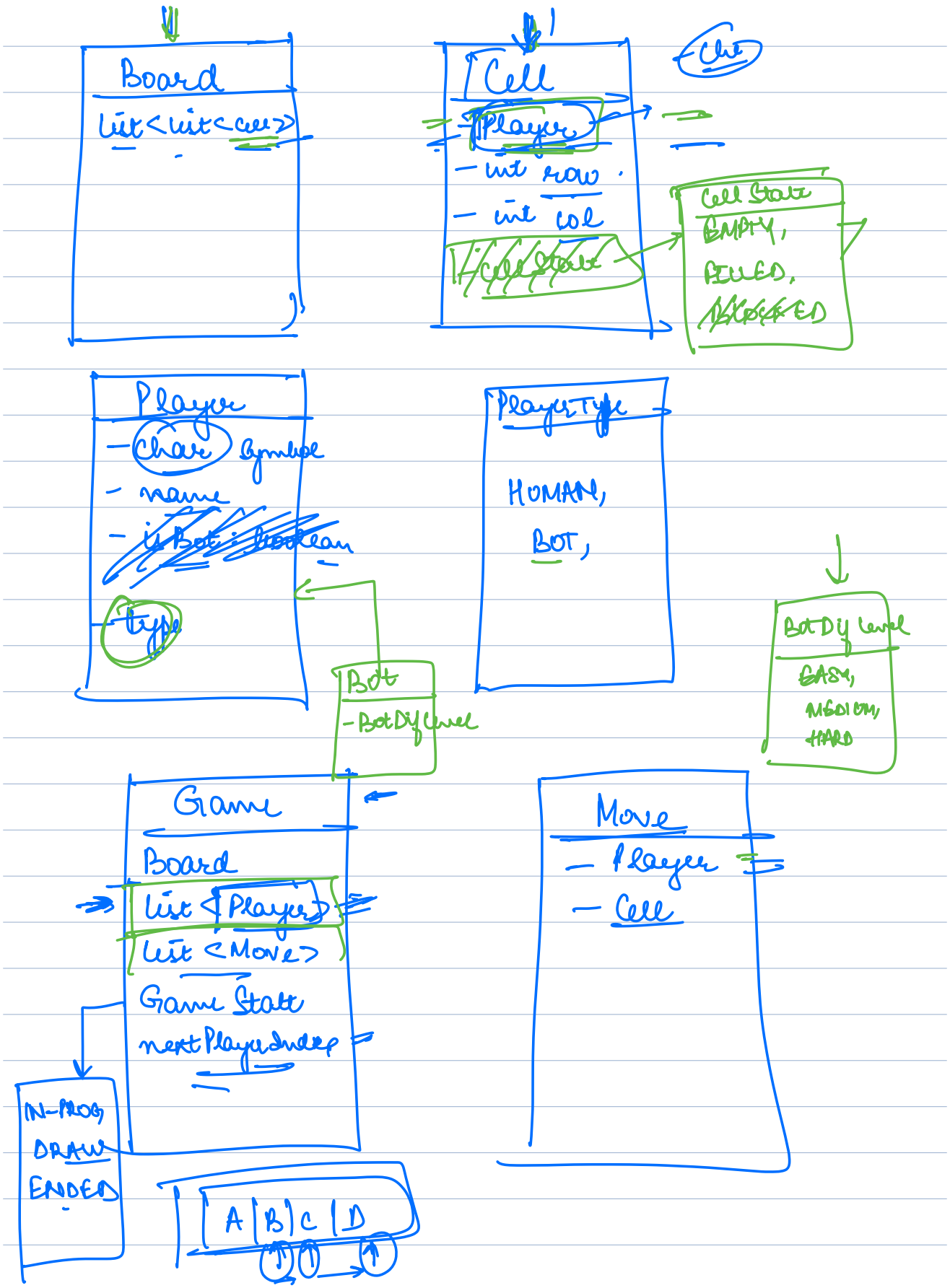
→ list all the entities that will be there

→ Nouns / visualization

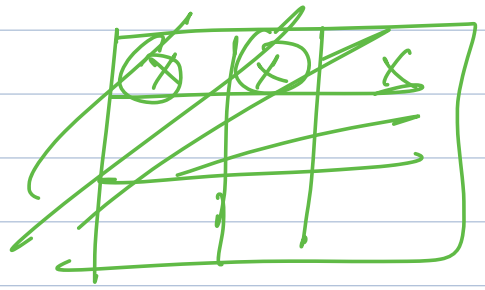
↑

Outer / Inner

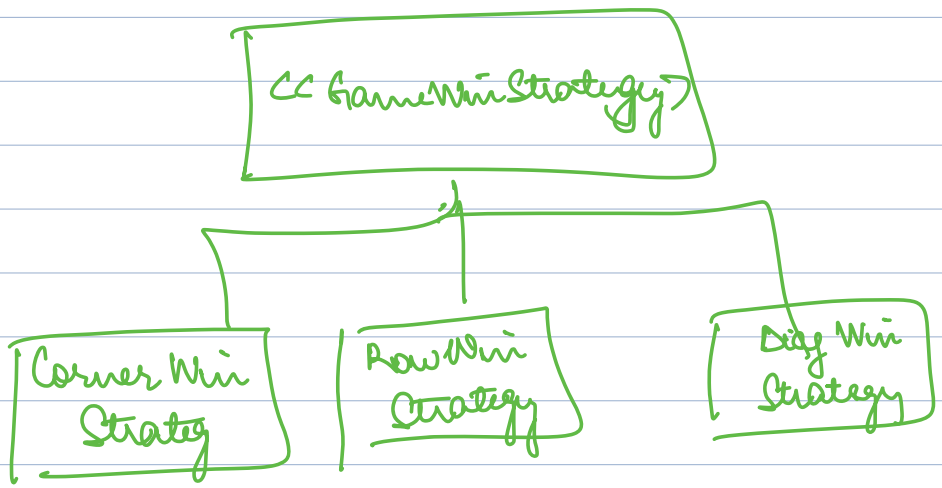


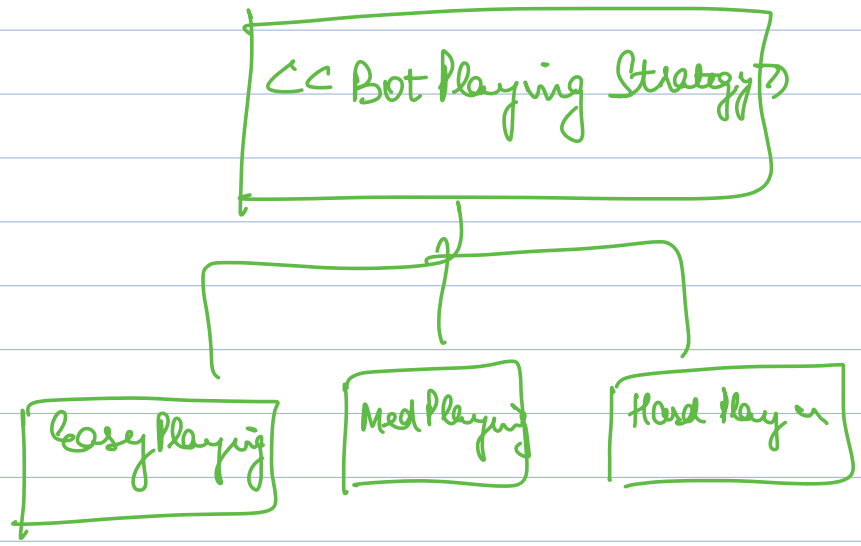

Board
List<List<Cell>>
Cell
Player
- int row
- int col
~~CellState~~
Cell State
EMPTY,
FILLED,
~~BLOCKED~~
Player
- Char symbol
- name
- ~~isBot : boolean~~
- Type
PlayerType
HUMAN,
BOT,
Bot
- BotDifLevel
BotDifLevel
EASY,
MEDIUM,
HARD
Game
Board
List<Player>
List<Move>
Game State
nextPlayerIndex
IN-PROG,
DRAW
ENDED
Move
- Player
- Cell
A B C D

Game Builder



Player factory

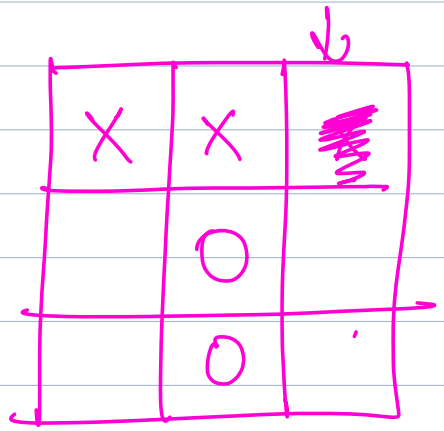


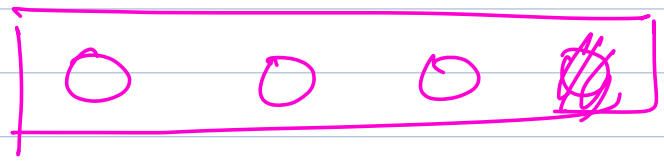



```
BotPlayingStrategyFactory {
    getStrategyForEnum(          ){

    }
}
```

How will we implement undo





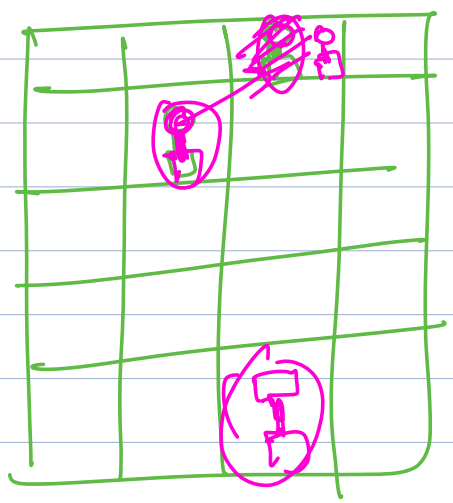
undo() {

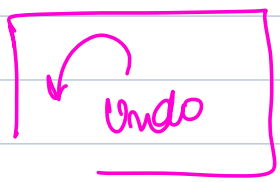
① remove from list move

② update the state

}

Opⁿ 1: Remove from list <move> and update the game state



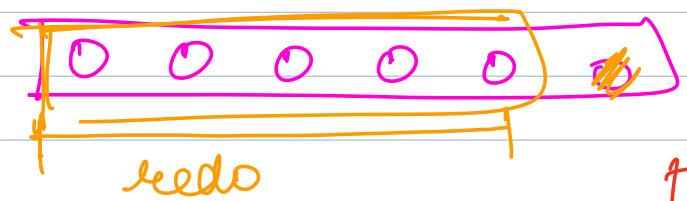


Loved Someone
They married someone else
Devastated

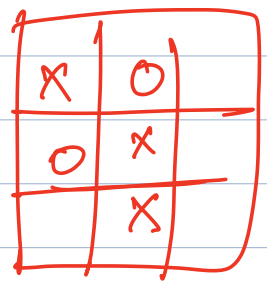
KKHH →

OSM →

$\boxed{Op^n 2}$ list <Move>

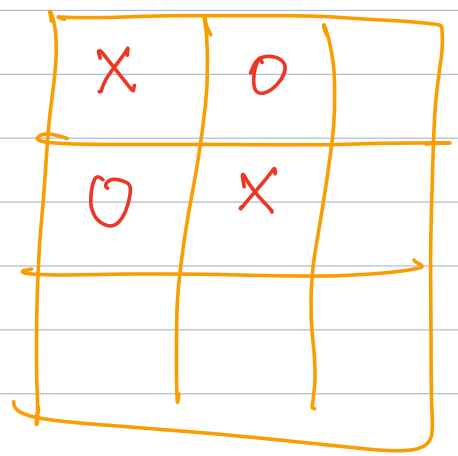


undo() {
Clear the state of game



}

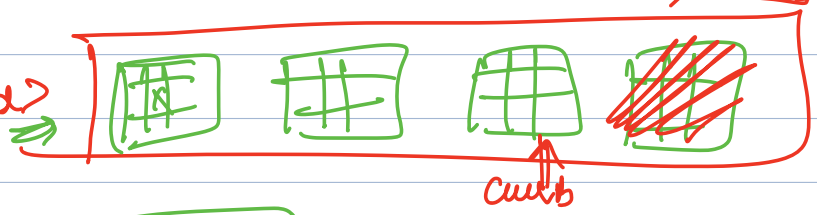
→ Chess etc
→ easy to imp
→ computationally expensive



Op^n Doraemon ⇒ Time Machine

List<Move>

List<Board>

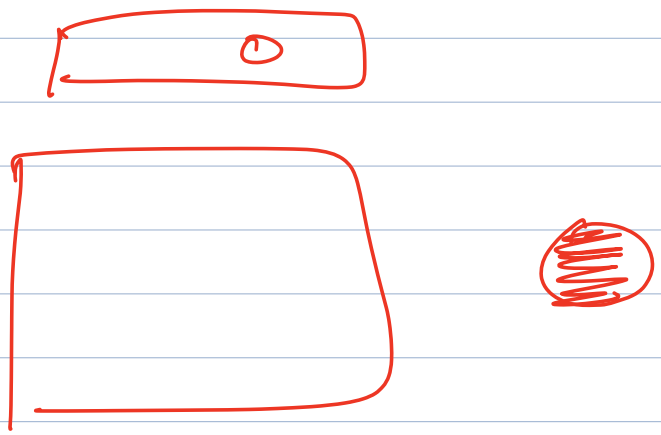


2B users → 9 By → 18 GB ⇒ RAM

user-id	boolean
8 By	1B

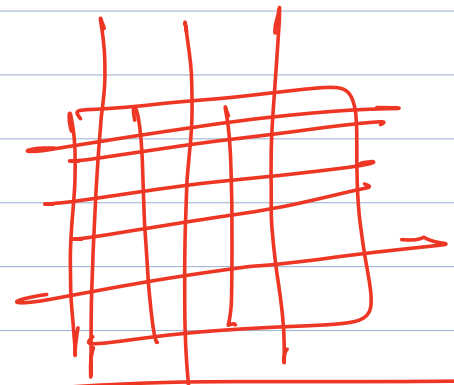
Pro ⇒ efficient

Con ⇒ Bit more space



H/W
1. Come up with similar class diagr
2. Code all the classes

③ Try to imp checking winner in $O(1)$
TC



$\Rightarrow O(N^2)$

$\Downarrow$

$O(1)$

Given Board and last Move check if someone has won in $O(1)$